

LAPORAN TUGAS BESAR STRATEGI ALGORITMA



Disusun oleh:

Imam Faris Rasyid (124140004)

Mifthahul Rezki Akbar (124140202)

Bimo Auliano (124140198)

Dosen Pengampu:

Imam Eko Wicaksono, S.Si., [M.Si.](#)

**TEKNIK INFORMATIKA
INSTITUT TEKNOLOGI SUMATERA
2026**

BAB I

Deskripsi Tugas

Robocode adalah permainan pemrograman yang bertujuan untuk membuat kode bot dalam bentuk tank virtual untuk berkompetisi melawan bot lain di arena. Pertempuran Robocode berlangsung hingga bot-bot bertarung hanya tersisa satu seperti permainan Battle Royale, karena itulah permainan ini dinamakan Tank Royale.

Dalam permainan ini, pemain berperan sebagai programmer bot dan tidak memiliki kendali langsung atas permainan. Pemain hanya bertugas untuk membuat program yang menentukan logika pada bot. Program yang dibuat akan berisi instruksi tentang cara bot bergerak, mendeteksi bot lawan, menembakkan senjatanya, serta bagaimana bot bereaksi terhadap berbagai kejadian selama pertempuran.

Pada tugas besar pertama ini, mahasiswa diminta untuk membuat sebuah bot yang nantinya akan dipertandingkan satu sama lain. Algoritma yang wajib digunakan untuk mengatur kemampuan bot adalah algoritma greedy, di mana strategi pengambilan keputusan harus berdasarkan pemilihan aksi terbaik secara lokal pada setiap langkah. Mahasiswa diminta untuk tidak hanya mengimplementasikan bot tersebut, tetapi juga menjelaskan strategi greedy yang digunakan dalam laporan ini secara rinci, mulai dari proses pemetaan persoalan ke elemen-elemen greedy hingga analisis efektivitas strategi tersebut.

Tujuan akhir dari tugas besar ini adalah untuk menghasilkan bot dengan strategi greedy terbaik yang mampu bersaing dengan bot dari kelompok lain dalam kompetisi. Selain implementasi teknis, laporan ini juga mencakup penjelasan teoritis, eksplorasi strategi alternatif, analisis efektivitas, serta pengujian terhadap solusi yang dibuat. Oleh karena itu, pemahaman mendalam tentang algoritma greedy dan kemampuan mengimplementasikannya ke dalam konteks permainan menjadi aspek utama yang diuji dalam tugas besar ini.

BAB II

Landasan Teori

2.1 Algoritma Greedy

Algoritma greedy merupakan salah satu strategi pemecahan masalah dalam bidang algoritma yang mengambil keputusan terbaik pada setiap langkah berdasarkan kondisi saat itu. Istilah greedy berarti “rakus”, karena algoritma ini selalu memilih pilihan yang dianggap paling menguntungkan secara lokal tanpa mempertimbangkan seluruh kemungkinan solusi di masa depan.

Prinsip utama dari algoritma greedy adalah memilih solusi lokal optimum pada setiap tahap dengan harapan bahwa rangkaian pilihan lokal tersebut akan menghasilkan solusi global yang optimum atau mendekati optimum. Dengan kata lain, algoritma greedy tidak mencoba semua kemungkinan solusi seperti algoritma *brute force*, melainkan langsung memilih kandidat terbaik berdasarkan kriteria tertentu.

Pada setiap langkah, algoritma greedy akan mengevaluasi himpunan kandidat yang tersedia, kemudian memilih kandidat yang paling baik menurut fungsi seleksi. Kandidat yang terpilih akan diperiksa kelayakannya menggunakan fungsi kelayakan. Jika kandidat tersebut memenuhi syarat, maka kandidat akan dimasukkan ke dalam himpunan solusi. Proses ini dilakukan secara berulang hingga solusi dianggap lengkap atau tidak ada lagi kandidat yang dapat dipilih.

Algoritma greedy memiliki beberapa kelebihan. Salah satu kelebihannya adalah sederhana dan mudah diimplementasikan. Selain itu, algoritma greedy umumnya memiliki waktu eksekusi yang lebih cepat dibandingkan algoritma yang mencoba semua kemungkinan solusi, karena greedy hanya mengambil satu keputusan terbaik pada setiap langkah. Hal ini membuat algoritma greedy cocok digunakan pada permasalahan yang membutuhkan pengambilan keputusan cepat.

Namun, algoritma greedy juga memiliki kelemahan. Karena hanya mempertimbangkan keputusan terbaik pada kondisi saat ini, algoritma greedy tidak selalu menghasilkan solusi yang paling optimal. Keputusan yang terlihat paling baik pada suatu tahap dapat menyebabkan hasil akhir yang kurang baik pada tahap berikutnya. Oleh karena itu, keberhasilan algoritma greedy sangat bergantung pada pemilihan kriteria seleksi yang digunakan.

2.2 Cara Kerja Program Secara Umum

Program yang dikembangkan dalam proyek ini bertujuan untuk menjalankan bot Orion dengan menggunakan pendekatan greedy *rule-based*. Program ini diimplementasikan dalam bentuk bot tank yang mampu bergerak, memindai posisi musuh menggunakan *radar*, mengarahkan *gun*, menembak, serta mengambil keputusan secara otomatis berdasarkan kondisi permainan seperti jarak musuh, energi, posisi arena, dan keadaan stuck.

2.3 Proses Scanning pada Bot

Bot secara berkala melakukan proses *scanning* atau pemindaian untuk mengetahui kondisi arena sekitarnya. Pemindaian ini dilakukan dalam radius tertentu untuk mendeteksi posisi musuh. Hasil *scanning* biasanya dikembalikan dalam *event* `ScannedBotEvent`. Informasi yang diperoleh dari hasil *scanning* ini kemudian menjadi dasar dalam proses pengambilan keputusan berikutnya. Bot mengevaluasi semua data yang terdeteksi untuk menentukan target mana yang akan dikejar terlebih dahulu.

2.4 Proses Pemilihan Aksi pada Bot

Setelah melakukan *scanning*, bot akan mengevaluasi semua data yang didapat berdasarkan beberapa kriteria, seperti jarak terdekat ke musuh, energi musuh, dan kondisi musuh saat ini. Dari hasil evaluasi tersebut, bot kemudian memilih aksi terbaik saat itu juga, mengikuti algoritma yang telah dibuat. Pemilihan ini tidak mempertimbangkan kondisi masa depan atau perencanaan jangka panjang.

2.5 Implementasi Algoritma Greedy

Strategi greedy dalam bot ini difokuskan pada ketepatan pengambilan keputusan dan pertimbangan optimalnya solusi yang dieksekusi. Bot tidak melakukan pencarian seluruh kemungkinan *state* di masa depan. Namun, bot tetap menggunakan prediksi sederhana untuk memperkirakan posisi musuh saat menembak. Keputusan selalu diambil berdasarkan kondisi arena saat ini yang diperoleh dari hasil *scanning*. Pendekatan ini cocok digunakan dalam arena permainan yang kondisinya selalu berubah-ubah. Namun, dalam beberapa kasus, algoritma greedy dapat mengarah pada solusi yang tidak optimal. Misalnya, bot tetap dapat mengalami posisi kurang menguntungkan ketika keputusan yang diambil tidak cukup baik berdasarkan kondisi arena saat itu, seperti terlalu dekat dengan dinding, terlalu fokus pada satu target, atau gagal menjaga jarak aman dari musuh.

BAB III

Aplikasi Strategi Greedy

3.1 Mapping Persoalan Robocode Tank Royale ke Elemen Algoritma Greedy

Dalam penyelesaian persoalan pertarungan pada Robocode Tank Royale, strategi greedy diaplikasikan dengan memetakan masalah ke dalam elemen-elemen algoritma greedy. Pada bot, setiap keputusan diambil berdasarkan kondisi permainan saat itu, seperti jarak musuh, energi bot, energi musuh, posisi bot terhadap dinding arena, serta kondisi pergerakan bot.

- Himpunan kandidat: Semua aksi yang dapat dipilih oleh bot pada kondisi permainan saat itu, seperti mendekati musuh, menjauhi musuh, menghindari menabrak dinding, mengarahkan *radar*, mengarahkan senjata, dan menembak.
- Himpunan solusi: Urutan aksi yang dipilih oleh bot selama pertandingan untuk bertahan hidup dan menyerang musuh, misalnya bergerak mendekati musuh saat jarak jauh, menjaga jarak saat jarak terlalu dekat, serta menembak ketika arah *gun* sudah tepat.
- Fungsi solusi: Aksi terbaik yang dipilih oleh bot pada setiap kondisi permainan. Contohnya adalah menentukan arah gerak bot, menentukan besar peluru, mengunci *radar* ke arah musuh, atau mengarahkan *gun* ke posisi musuh yang diprediksi.
- Fungsi seleksi: Pemilihan aksi berdasarkan kondisi lokal saat itu, seperti jarak musuh, energi bot, energi musuh, posisi bot terhadap dinding arena, kondisi stuck, dan arah musuh. Misalnya, jika musuh jauh maka bot mendekat, jika jarak sedang maka bot mengorbit, dan jika terlalu dekat maka bot mundur.
- Fungsi kelayakan: Pemeriksaan apakah suatu aksi layak dilakukan. Contohnya, bot hanya menembak jika *gun* sudah cukup mengarah ke target, selisih sudut *gun* masih berada dalam batas toleransi, dan GunHeat bernilai 0, menjauh dari dinding jika dekat dinding atau stuck, serta memilih besar peluru sesuai energi dan jarak musuh.
- Fungsi objektif: Memaksimalkan peluang bot untuk bertahan hidup dan mengalahkan musuh dengan cara menjaga jarak aman, menghindari dinding atau stuck, menghemat energi tembakan, serta menembak ketika peluang mengenai musuh lebih besar.

3.2 Alternatif Strategi Greedy pada Robocode Tank Royale

Strategi greedy dapat diterapkan dengan menggunakan beberapa heuristic yang berbeda. Setiap heuristic menentukan prioritas pengambilan keputusan bot berdasarkan kondisi permainan saat itu.

Strategi pertama diterapkan pada bot AllMight. Bot ini menggunakan pendekatan greedy dengan memilih target berdasarkan musuh yang sedang dikunci atau musuh yang memiliki jarak lebih dekat. Keputusan bot dipengaruhi oleh jumlah musuh yang tersisa, jarak

terhadap target, energi musuh, posisi arena, serta kondisi darurat seperti terkena peluru atau menabrak dinding.

Heuristic utama pada strategi ini adalah pemilihan aksi berdasarkan target terkunci dan kondisi pertempuran. Jika musuh banyak, bot cenderung bermain lebih aman dengan menjaga jarak dan menghindari pertempuran langsung. Jika jumlah musuh semakin sedikit, bot menjadi lebih agresif dengan mendekati target dan meningkatkan peluang serangan. Bot juga dapat menabrak bot secara sengaja apabila musuh memiliki energi rendah atau berada pada jarak sangat dekat.

Strategi ini termasuk greedy karena pada setiap *turn* bot memilih aksi yang dianggap paling menguntungkan berdasarkan kondisi saat itu, tanpa mengevaluasi seluruh kemungkinan aksi pada giliran berikutnya.

Strategi kedua diterapkan pada bot Orion. Bot ini menggunakan pendekatan greedy *rule-based*, yaitu memilih aksi berdasarkan aturan kondisi yang telah ditentukan. Keputusan utama bot dipengaruhi oleh jarak musuh, energi bot, energi musuh, posisi terhadap dinding arena, dan kondisi stuck.

Heuristic utama pada strategi ini adalah pemilihan aksi berdasarkan jarak dan keamanan posisi. Jika musuh berada pada jarak jauh, bot akan mendekat dengan sudut tertentu. Jika musuh berada pada jarak sedang, bot akan melakukan orbit untuk menjaga posisi saat melakukan penembakan. Jika musuh terlalu dekat, bot akan mundur agar tidak mudah tertabrak atau terkena serangan langsung. Selain itu, jika bot berada dekat dinding atau mengalami stuck, bot akan memprioritaskan menuju tengah arena.

Bot ini juga menggunakan pemilihan kekuatan tembakan secara greedy berdasarkan jarak dan energi. Semakin dekat musuh, semakin besar daya tembak yang digunakan. Namun, jika energi bot rendah, daya tembak akan dikurangi agar bot tidak cepat kehabisan energi. Strategi ini termasuk greedy karena bot selalu memilih aksi terbaik berdasarkan kondisi saat itu.

Strategi ketiga diterapkan pada bot Aegis. Aegis menggunakan strategi greedy defensif yang berfokus pada bertahan hidup dan menjaga jarak aman. Bot memilih aksi berdasarkan posisi terhadap dinding, jarak musuh, energi bot, dan kondisi *gun*. Jika bot berada dekat dinding, Aegis akan bergerak menuju tengah arena. Jika musuh terlalu dekat, bot bergerak menjauh dengan sudut besar. Jika jarak musuh sedang atau jauh, bot bergerak menyamping agar lebih sulit ditembak.

Heuristic utama Aegis adalah defensive survival movement. Bot lebih mengutamakan posisi aman dibanding mengejar damage besar. Aegis hanya menembak ketika musuh berada cukup dekat, energi bot masih aman, arah *gun* sudah cukup akurat, dan GunHeat bernilai 0. Strategi ini termasuk greedy karena pada setiap *turn* bot langsung memilih aksi yang dianggap paling aman dan menguntungkan berdasarkan kondisi saat itu.

Strategi keempat diterapkan pada bot Ares. Ares menggunakan strategi greedy agresif yang berfokus pada ramming. Bot ini memilih aksi paling langsung berdasarkan kondisi saat

itu, yaitu mengarahkan body ke posisi musuh, maju ke arah musuh, dan menembak ketika *gun* sudah cukup mengarah.

Heuristic utama pada strategi ini adalah aggressive ramming. Jika musuh terdeteksi, bot langsung mendekat untuk menabrak musuh tanpa melakukan orbit atau menjaga jarak. Strategi ini bertujuan untuk memperoleh ram damage dan memberikan tekanan terus-menerus kepada lawan. Selain itu, Ares tetap memilih besar peluru berdasarkan jarak dan energi agar serangan tidak terlalu boros.

Strategi ini termasuk greedy karena pada setiap *turn* bot memilih aksi yang dianggap paling menguntungkan secara langsung, yaitu mendekati musuh, menabrak, dan menembak. Bot tidak mengevaluasi seluruh kemungkinan posisi musuh pada beberapa *turn* berikutnya, melainkan mengambil keputusan berdasarkan posisi musuh yang terdeteksi saat itu.

3.3 Analisis Efisiensi dan Efektivitas Alternatif Strategi Greedy

Pada strategi greedy Orion, pendekatan *rule-based* memberikan efisiensi yang baik karena bot hanya mengevaluasi kondisi saat ini seperti jarak musuh, energi bot, energi musuh, posisi terhadap dinding, dan kondisi stuck. Perhitungan yang dilakukan relatif ringan sehingga sesuai untuk permainan real-time yang memiliki batas waktu *turn*. Bot tidak melakukan pencarian seluruh kemungkinan aksi, melainkan langsung memilih aksi berdasarkan aturan prioritas yang telah ditentukan.

Dari sisi efektivitas, Orion cukup seimbang karena mampu menyerang dan bertahan. Pemilihan besar peluru berdasarkan jarak dan energi membuat penggunaan energi lebih terkendali. Pergerakan orbit membantu bot tetap bergerak saat menyerang, sementara mekanisme wall avoidance dan stuck recovery membantu bot keluar dari posisi berbahaya. Selain itu, prediksi sederhana terhadap posisi musuh membantu meningkatkan peluang tembakan mengenai target. Namun, strategi ini tetap memiliki kelemahan karena keputusan yang diambil bersifat lokal. Dalam kondisi tertentu, pilihan yang terlihat baik pada *turn* saat itu belum tentu menghasilkan posisi terbaik pada *turn* berikutnya.

Pada strategi AllMight, membawakan pendekatan terhadap keputusan bot berdasarkan keadaan pada medan pertempuran merupakan opsi yang baik, namun pada penerapannya efisiensi dan efektivitas dari strategi tersebut tidak berjalan baik, terlebih lagi di situasi 1 vs 1.

Dimana algoritma pada strategi ini dirancang untuk bereaksi terhadap kondisi medan pertempuran, jarak terhadap target, energi musuh dan kondisi darurat pada bot itu sendiri. dimana jika segmen yang digunakan adalah battle royale, aspek aspek tersebut tidak diperlukan secara langsung, pendekatan yang lebih linear dan aktif seperti menekan musuh akan jauh lebih unggul.

Pada strategi greedy yang diterapkan pada Aegis, pendekatan defensif memberikan efisiensi komputasi yang baik karena bot hanya mengevaluasi beberapa kondisi sederhana, yaitu jarak musuh, posisi bot terhadap dinding, energi bot, dan kondisi *gun*. Bot tidak melakukan prediksi kompleks atau pencarian kemungkinan *state* di masa depan, sehingga

proses pengambilan keputusan menjadi ringan dan cepat untuk dijalankan pada game real-time.

Dari sisi efektivitas, Aegis cukup baik untuk menjaga kelangsungan hidup karena bot berusaha menjauh dari dinding dan menjaga jarak dari musuh. Jika bot berada dekat dinding, bot akan bergerak menuju tengah arena. Jika musuh terlalu dekat, bot bergerak menjauh dengan sudut besar. Namun, strategi ini kurang efektif untuk mengejar skor damage karena bot hanya menembak dengan peluru kecil dan hanya saat kondisi cukup aman. Pada pertarungan 1 vs 1, strategi yang terlalu defensif dapat membuat bot kesulitan memberikan tekanan kepada lawan.

Pada strategi greedy Ares, efisiensi algoritma sangat tinggi karena keputusan yang diambil sangat sederhana. Bot hanya perlu mendeteksi posisi musuh, mengarahkan body ke arah musuh, maju untuk menabrak, dan menembak jika *gun* cukup mengarah. Karena tidak menggunakan banyak kondisi kompleks, strategi ini ringan secara komputasi dan mudah dijalankan dalam batas waktu *turn*.

Namun, efektivitas Ares sangat bergantung pada situasi. Dalam duel jarak dekat, strategi ramming dapat memberikan tekanan besar dan berpotensi menghasilkan ram damage. Akan tetapi, pada kondisi musuh bergerak cepat atau ketika bot berada dekat dinding, Ares dapat kehilangan energi karena terlalu sering menabrak atau terlalu mudah ditembak. Strategi ini agresif, tetapi kurang optimal dibandingkan Orion.

3.4 Strategi Greedy yang Dipilih dari Seluruh Alternatif

Berdasarkan perbandingan keempat alternatif strategi greedy, strategi Orion dipilih sebagai strategi utama karena memiliki keseimbangan paling baik antara menyerang, bertahan, dan menjaga efisiensi energi. Orion tidak terlalu agresif seperti Ares, tidak terlalu defensif seperti Aegis, dan lebih mudah dikontrol. Strategi greedy yang dipilih pada bot Orion adalah:

- Bot menentukan aksi berdasarkan kondisi arena secara langsung menggunakan pendekatan *rule-based*
- Jika musuh berada pada jarak jauh, bot akan mendekati musuh dengan sudut tertentu untuk memperoleh posisi serang yang lebih baik.
- Jika musuh berada pada jarak sedang, bot akan melakukan orbit movement sambil melakukan penembakan untuk menjaga mobilitas dan akurasi serangan.
- Jika musuh berada terlalu dekat, bot akan mundur untuk menghindari tabrakan atau serangan langsung dari lawan.
- Jika posisi bot terlalu dekat dengan dinding arena atau mengalami stuck, bot akan bergerak menuju bagian tengah arena untuk memperoleh ruang gerak yang lebih aman.

- Bot menentukan kekuatan tembakan berdasarkan jarak musuh dan energi yang dimiliki. Semakin dekat musuh, semakin besar daya tembak yang digunakan. Namun jika energi bot rendah, daya tembak akan dikurangi untuk menjaga efisiensi energi.

Pertimbangan pemilihan strategi ini adalah:

- Pendekatan *rule-based* lebih sederhana untuk diimplementasikan dan mudah dikontrol.
- Pengambilan keputusan dapat dilakukan dengan cepat karena bot hanya memproses kondisi saat ini tanpa melakukan pencarian kemungkinan *state* di masa depan.
- Strategi orbit, retreat, dan wall avoidance membantu meningkatkan survivability bot selama pertarungan berlangsung.
- Pengaturan daya tembak berdasarkan kondisi energi membuat penggunaan energi menjadi lebih efisien.
- Strategi greedy ini cukup efektif untuk game real-time karena mampu menjaga keseimbangan antara pergerakan, pertahanan, dan serangan secara responsif.

BAB IV

IMPLEMENTASI DAN PENGUJIAN

4.1. Implementasi

4.1.1 Pemilihan Besar Peluru pada Bot Orion

FUNCTION choose_fire_power(enemy_distance, bot_energy, enemy_energy):

IF enemy_distance < 120 THEN

 fire_power ← 3.0

ELSE IF enemy_distance < 300 THEN

 fire_power ← 2.0

ELSE IF enemy_distance < 550 THEN

 fire_power ← 1.1

ELSE

 fire_power ← 0.7

END IF

// Jika energi bot rendah, kurangi besar peluru

IF bot_energy < 30 AND fire_power > 1.0 THEN

 fire_power ← 1.0

END IF

IF bot_energy < 15 AND fire_power > 0.7 THEN

 fire_power ← 0.7

END IF

// Jika energi musuh rendah, peluru besar tidak diperlukan

IF enemy_energy < 15 AND fire_power > 1.0 THEN

 fire_power ← 1.0

END IF

// Jika bot unggul energi dan musuh dekat, gunakan peluru besar

IF enemy_distance < 120 AND bot_energy > enemy_energy + 20 THEN

 fire_power ← 3.0

END IF

// Pastikan fire power berada dalam batas aman

IF fire_power < 0.1 THEN

 fire_power ← 0.1

ELSE IF fire_power > 3.0 THEN

 fire_power ← 3.0

END IF

```
RETURN fire_power
```

```
END FUNCTION
```

4.1.2 Prediksi Posisi Musuh pada Bot Orion

```
FUNCTION predict_enemy_position(enemy_x, enemy_y, enemy_direction,  
enemy_speed, enemy_distance):
```

```
IF enemy_distance < 120 THEN  
    prediction_time ← 0.5  
ELSE IF enemy_distance < 300 THEN  
    prediction_time ← 1.0  
ELSE IF enemy_distance < 550 THEN  
    prediction_time ← 1.5  
ELSE  
    prediction_time ← 2.0  
END IF
```

```
// Ubah arah gerak musuh dari derajat ke radian  
enemy_move_direction ← enemy_direction × PI / 180
```

```
predicted_x ← enemy_x + SIN(enemy_move_direction) × enemy_speed ×  
prediction_time  
predicted_y ← enemy_y + COS(enemy_move_direction) × enemy_speed ×  
prediction_time
```

```
// Batasi posisi prediksi agar tetap berada di dalam arena  
IF predicted_x < 0 THEN  
    predicted_x ← 0  
ELSE IF predicted_x > arena_width THEN  
    predicted_x ← arena_width  
END IF
```

```
IF predicted_y < 0 THEN  
    predicted_y ← 0  
ELSE IF predicted_y > arena_height THEN  
    predicted_y ← arena_height  
END IF
```

```
RETURN (predicted_x, predicted_y)
```

```
END FUNCTION
```

4.1.3 Pemilihan Gerakan Bot Orion

FUNCTION choose_movement(enemy_distance, enemy_direction, bot_energy, enemy_energy, is_stuck, is_near_wall):

IF is_stuck = TRUE THEN

target_direction $\leftarrow$ direction_to_arena_center

move_distance $\leftarrow$ 180

// Jika bot stuck, prioritas utama adalah keluar dari posisi tersebut

ELSE IF is_near_wall = TRUE THEN

orbit_direction $\leftarrow$ orbit_direction $\times$ -1

target_direction $\leftarrow$ direction_to_arena_center

move_distance $\leftarrow$ 180

// Jika dekat dinding, bot bergerak ke tengah arena

ELSE IF bot_energy > enemy_energy + 25

AND bot_energy > 40

AND enemy_energy < 30

AND enemy_distance < 180 THEN

target_direction $\leftarrow$ enemy_direction

move_distance $\leftarrow$ enemy_distance

// Jika bot unggul energi dan musuh lemah, bot menekan musuh

ELSE IF enemy_distance > 250 THEN

target_direction $\leftarrow$ enemy_direction + orbit_direction $\times$ 45

move_distance $\leftarrow$ 180

// Jika musuh jauh, bot mendekat sambil sedikit mengorbit

ELSE IF enemy_distance > 120 THEN

target_direction $\leftarrow$ enemy_direction + orbit_direction $\times$ 90

move_distance $\leftarrow$ 120

// Jika jarak sedang, bot mengorbit musuh

ELSE

target_direction $\leftarrow$ enemy_direction + 180

move_distance $\leftarrow$ 120

// Jika terlalu dekat, bot menjauh dari musuh

END IF

RETURN (target_direction, move_distance)

END FUNCTION

4.1.4 Deteksi Stuck pada Bot Orion

FUNCTION check_stuck(current_x, current_y, previous_x, previous_y, distance_remaining, max_speed):

move_x $\leftarrow$ current_x - previous_x

move_y $\leftarrow$ current_y - previous_y

IF ABS(move_x) < 1 AND ABS(move_y) < 1 AND distance_remaining > max_speed THEN

stuck_turns $\leftarrow$ stuck_turns + 1

ELSE

stuck_turns $\leftarrow$ 0

END IF

previous_x $\leftarrow$ current_x

previous_y $\leftarrow$ current_y

IF stuck_turns $\geq$ 3 THEN

is_stuck $\leftarrow$ TRUE

orbit_direction $\leftarrow$ orbit_direction $\times$ -1

stuck_turns $\leftarrow$ 0

ELSE

is_stuck $\leftarrow$ FALSE

END IF

RETURN is_stuck

END FUNCTION

4.1.5 Keputusan Menembak pada Bot Orion

FUNCTION should_fire(enemy_distance, gun_turn, gun_heat):

IF enemy_distance $\leq$ 150 THEN

aim_tolerance $\leftarrow$ 6

ELSE IF enemy_distance $\leq$ 350 THEN

aim_tolerance $\leftarrow$ 4

ELSE

aim_tolerance $\leftarrow$ 2

END IF

```
// Tembak hanya jika gun cukup akurat dan tidak panas
IF ABS(gun_turn) < aim_tolerance AND gun_heat = 0 THEN
    RETURN TRUE
ELSE
    RETURN FALSE
END IF
```

END FUNCTION

4.1.6 Alur Utama Saat Musuh Terdeteksi pada Bot Orion

FUNCTION on_scanned_bot(enemy):

```
    enemy_distance ← distance_to(enemy)
    enemy_direction ← direction_to(enemy)

    lock_radar_to(enemy)

    is_stuck ← check_stuck(current_position, previous_position)

    fire_power ← choose_fire_power(enemy_distance, bot_energy, enemy_energy)

    predicted_position ← predict_enemy_position(enemy)

    gun_turn ← turn_gun_to(predicted_position)

    is_near_wall ← check_near_wall(bot_position)

    (move_direction, move_distance) ← choose_movement(
        enemy_distance,
        enemy_direction,
        bot_energy,
        enemy_energy,
        is_stuck,
        is_near_wall
    )

    turn_body_to(move_direction)
    move_forward(move_distance)

    IF should_fire(enemy_distance, gun_turn, gun_heat) THEN
        fire(fire_power)
    END IF
```

```
execute_all_actions()
```

```
END FUNCTION
```

4.1.7 Strategi Utama Ares Saat Musuh Terdeteksi pada Bot Ares

FUNCTION on_scanned_bot(enemy):

```
    enemy_distance ← distance_to(enemy)
```

```
    lock_radar_to(enemy)
```

```
    // Radar diarahkan ke musuh agar posisi musuh tetap terpantau.
```

```
    gun_turn ← turn_gun_to(enemy)
```

```
    // Gun diarahkan langsung ke posisi musuh.
```

```
    turn_body_to(enemy)
```

```
    // Body diarahkan langsung ke musuh untuk melakukan ramming.
```

```
    move_forward(enemy_distance)
```

```
    // Bot selalu maju ke arah musuh agar dapat menabrak.
```

```
    fire_power ← choose_fire_power(enemy_distance, bot_energy)
```

```
    // Besar peluru dipilih berdasarkan jarak dan energi bot.
```

```
    IF ABS(gun_turn) < 10 AND gun_heat = 0 THEN
```

```
        fire(fire_power)
```

```
        // Bot menembak hanya jika gun cukup mengarah dan tidak panas.
```

```
    END IF
```

```
    execute_all_actions()
```

```
END FUNCTION
```

4.1.8 Pemilihan Besar Peluru pada Bot Ares

FUNCTION choose_fire_power(enemy_distance, bot_energy):

```
    IF enemy_distance < 120 THEN
```

```
        fire_power ← 3.0
```

```
    ELSE IF enemy_distance < 300 THEN
```

```
        fire_power ← 2.0
```

```
    ELSE
```

```
        fire_power ← 1.0
```

```
    END IF
```

```
// Jika energi bot rendah, peluru dikurangi agar tidak boros energi.  
IF bot_energy < 30 AND fire_power > 1.0 THEN  
    fire_power ← 1.0  
END IF
```

```
IF bot_energy < 15 AND fire_power > 0.7 THEN  
    fire_power ← 0.7  
END IF
```

```
RETURN fire_power
```

```
END FUNCTION
```

4.1.9 Strategi Bot Ares Saat Menabrak Bot Musuh

FUNCTION on_hit_bot(enemy):

```
    turn_body_to(enemy)  
    // Body tetap diarahkan ke musuh yang tertabrak.  
  
    move_forward(distance_to(enemy))  
    // Bot tetap maju agar terus memberi tekanan melalui ramming.  
  
    gun_turn ← turn_gun_to(enemy)  
    // Gun diarahkan ke musuh yang tertabrak.  
  
    IF ABS(gun_turn) < 10 AND gun_heat = 0 THEN  
        fire(2.0)  
        // Saat jarak sangat dekat, bot menembak dengan power sedang.  
    END IF
```

```
    execute_all_actions()
```

```
END FUNCTION
```

4.1.10 Strategi Bot Ares Saat Menabrak Dinding

FUNCTION on_hit_wall():

```
    turn_body_to(arena_center)  
    move_forward(180)  
  
    // Jika menabrak dinding, bot kembali ke tengah arena agar tidak tersangkut.  
  
    execute_all_actions()
```


END FUNCTION

4.1.11 Strategi Bot Ares Saat Terkena Peluru

FUNCTION on_hit_by_bullet():

 move_forward(120)

 // Saat terkena peluru, bot tetap bergerak agar tidak diam.

 execute_all_actions()

END FUNCTION

4.1.12 Strategi Pemilihan Target pada Bot AllMight

FUNCTION choose_target(enemy):

 distance $\leftarrow$ distance_to(enemy)

 IF enemy is current target OR no target is locked OR distance < locked_distance - 120 THEN

 target_id $\leftarrow$ enemy.id

 locked_distance $\leftarrow$ distance

 last_scan_turn $\leftarrow$ current_turn

 // Musuh dipilih jika merupakan target lama, belum ada target, atau jauh lebih dekat.

 END IF

END FUNCTION

4.1.13 Strategi Pergerakan Bot AllMight

FUNCTION choose_movement(enemy_distance, enemy_energy, enemy_count):

 IF enemy_count < 3 OR enemy_distance $\leq$ 100 AND enemy_energy < 60 THEN

 move_angle $\leftarrow$ direction_to_enemy_prediction

 // Jika musuh sedikit atau sangat dekat dan lemah, bot menabrak musuh.

 ELSE IF enemy_count > 2 THEN

 IF enemy_distance > 350 THEN

 move_angle $\leftarrow$ direction_to_enemy_prediction + 40 $\times$ orbit_direction

 ELSE IF enemy_distance > 200 THEN

 move_angle $\leftarrow$ direction_to_enemy_prediction + 80 $\times$ orbit_direction

 ELSE

```
        move_angle ← direction_to_enemy_prediction + 130 × orbit_direction
    END IF
    // Saat musuh banyak, bot lebih berhati-hati dan menjaga jarak.

ELSE
    IF enemy_distance > 150 THEN
        move_angle ← direction_to_enemy_prediction + 30 × orbit_direction
    ELSE
        move_angle ← direction_to_enemy_prediction + 45 × orbit_direction
    END IF
    // Saat musuh sedikit, bot menjadi lebih agresif.
END IF

move_forward(150)

END FUNCTION
```

4.1.14 Strategi Menembak Bot AllMight

FUNCTION choose_fire(enemy_distance, bot_energy, enemy_count):

```
    IF enemy_distance ≤ 150 THEN
        fire_power ← 3.0
    ELSE IF enemy_distance ≤ 300 THEN
        fire_power ← 2.0
    ELSE IF enemy_distance ≤ 450 THEN
        fire_power ← 1.0
    ELSE
        fire_power ← 0
    END IF

    IF enemy_count > 2 AND bot_energy < 40 AND enemy_distance > 200 THEN
        fire_power ← 0
        // Jika energi rendah dan musuh masih banyak, bot menahan tembakan.
    END IF

    IF bot_energy < 10 THEN
        fire_power ← MIN(fire_power, 0.1)
        // Jika energi sangat rendah, bot hanya memakai peluru kecil.
    END IF

    IF fire_power > 0 AND gun_heat = 0 THEN
        aim_gun_to_enemy_prediction()
        fire(fire_power)
    END IF
```

END FUNCTION

4.1.15 Strategi Utama Aegis Saat Musuh Terdeteksi

FUNCTION on_scanned_bot(enemy):

 enemy_distance $\leftarrow$ distance_to(enemy)

 lock_radar_to(enemy)

 // Radar diarahkan ke musuh agar posisi musuh tetap terpantau.

 gun_turn $\leftarrow$ turn_gun_to(enemy)

 // Gun diarahkan langsung ke posisi musuh.

 IF bot_is_near_wall THEN

 move_direction $\leftarrow$ direction_to_arena_center

 move_distance $\leftarrow$ 180

 // Jika dekat dinding, bot bergerak ke tengah arena.

 ELSE IF enemy_distance < 180 THEN

 move_direction $\leftarrow$ direction_to(enemy) + 140

 move_distance $\leftarrow$ 180

 // Jika musuh terlalu dekat, bot menjauh dengan sudut besar.

 ELSE IF enemy_distance < 350 THEN

 move_direction $\leftarrow$ direction_to(enemy) + 100

 move_distance $\leftarrow$ 150

 // Jika jarak sedang, bot bergerak menyamping untuk menghindari.

 ELSE

 move_direction $\leftarrow$ direction_to(enemy) + 90

 move_distance $\leftarrow$ 120

 // Jika musuh jauh, bot tetap bergerak menyamping agar sulit ditembak.

 END IF

 turn_body_to(move_direction)

 move_forward(move_distance)

 IF enemy_distance < 200 AND bot_energy > 45 AND ABS(gun_turn) < 6 AND
 gun_heat = 0 THEN

```
    fire(0.8)
    // Aegis hanya menembak saat kondisi cukup aman.
END IF
```

```
execute_all_actions()
```

```
END FUNCTION
```

4.1.16 Strategi Bot Aegis Saat Terkena Peluru

FUNCTION on_hit_by_bullet():

```
    turn_body_left(100)
    move_forward(180)
```

```
    // Saat terkena peluru, bot segera bergerak menjauh agar tidak diam di posisi yang sama.
```

```
    execute_all_actions()
```

```
END FUNCTION
```

4.1.17 Strategi Bot Aegis Saat Menabrak Dinding

FUNCTION on_hit_wall():

```
    move_direction ← direction_to_arena_center
    move_distance ← 220
```

```
    // Jika menabrak dinding, bot langsung diarahkan ke tengah arena.
```

```
    turn_body_to(move_direction)
    move_forward(move_distance)
```

```
    execute_all_actions()
```

```
END FUNCTION
```

4.1.18 Strategi Bot Aegis Saat Menabrak Bot Musuh

FUNCTION on_hit_bot():

```
    move_direction ← direction_to_arena_center
    move_distance ← 160
```

```
    // Jika menabrak bot lain, Aegis tidak melanjutkan ramming.
    // Bot memilih kembali ke tengah arena agar posisi lebih aman.
```

```
turn_body_to(move_direction)
move_forward(move_distance)

execute_all_actions()
```

END FUNCTION

4.2. Struktur Data

4.2.1 Struktur Data yang Digunakan

4.2.1.1 Kelas Orion

Kelas Orion merupakan kelas utama yang merepresentasikan bot pada permainan Robocode Tank Royale. Kelas ini mewarisi (inherit) kelas Bot dari library Robocode sehingga dapat mengakses fungsi pergerakan, *radar*, senjata, dan *event* permainan.

Kelas Orion bertanggung jawab untuk menjalankan seluruh strategi *greedy rule-based* yang telah dirancang. Setiap kali bot memperoleh informasi baru dari hasil *scanning* atau *event* tertentu, kelas ini akan memproses data tersebut dan memilih aksi yang dianggap paling menguntungkan pada kondisi saat itu.

Kelas ini digunakan untuk:

- Mengatur strategi pergerakan bot
- Mengatur sistem penembakan
- Mengelola *radar* dan deteksi musuh
- Menangani *event* seperti terkena peluru, menabrak dinding, atau bertabrakan dengan musuh

Atribut:

- OrbitDirection : int

Atribut orbitDirection digunakan untuk menyimpan arah orbit bot terhadap musuh. Nilai dari atribut ini biasanya berupa 1 atau -1. Nilai 1 menunjukkan bahwa bot bergerak mengorbit ke satu arah tertentu, sedangkan nilai -1 menunjukkan bahwa bot bergerak mengorbit ke arah sebaliknya. Atribut ini penting karena bot Orion menggunakan strategi orbit movement pada jarak menengah. Dengan bergerak menyamping atau mengorbit, bot menjadi lebih sulit ditembak oleh musuh dibandingkan jika hanya bergerak lurus. Selain itu, nilai orbitDirection dapat dibalik ketika bot mendeteksi kondisi tertentu, misalnya saat berada dekat dinding atau mengalami stuck. Pembalikan arah orbit membantu bot keluar dari pola gerak yang berpotensi membuatnya terjebak.

- prevX : double

Atribut *prevX* digunakan untuk menyimpan posisi bot pada sumbu X dari *turn* sebelumnya. Data ini diperlukan dalam proses deteksi stuck. Dengan membandingkan posisi X saat ini dan posisi X sebelumnya, bot dapat mengetahui apakah dirinya benar-benar berpindah atau hanya diam di tempat. Jika perubahan posisi sangat kecil dalam beberapa *turn*, sementara perintah gerak masih diberikan, maka bot dapat dianggap mengalami stuck.

- *prevY* : double

Atribut *prevY* digunakan untuk menyimpan posisi bot pada sumbu Y dari *turn* sebelumnya. Sama seperti *prevX*, atribut ini digunakan untuk mendeteksi apakah bot mengalami perpindahan posisi secara normal atau tidak. Deteksi stuck tidak cukup hanya menggunakan satu sumbu koordinat. Oleh karena itu, perubahan posisi pada sumbu X dan Y perlu diperiksa secara bersamaan agar hasil deteksi lebih akurat.

- *hasPrevPosition* : bool

Atribut *hasPrevPosition* digunakan sebagai penanda apakah posisi sebelumnya sudah pernah disimpan. Pada awal permainan, bot belum memiliki data posisi sebelumnya, sehingga perbandingan antara posisi saat ini dan posisi sebelumnya belum dapat dilakukan. Jika *hasPrevPosition* bernilai false, maka bot akan menyimpan posisi saat ini sebagai posisi awal. Setelah itu, nilai *hasPrevPosition* diubah menjadi true, sehingga pada *turn* berikutnya bot sudah dapat melakukan perbandingan posisi.

- *stuckTurns* : int

Atribut *stuckTurns* digunakan untuk menghitung jumlah *turn* ketika bot terdeteksi tidak bergerak secara normal. Bot dianggap tidak bergerak normal apabila perubahan posisinya sangat kecil, padahal masih terdapat perintah gerakan yang belum selesai. Jika nilai *stuckTurns* mencapai batas tertentu, misalnya tiga *turn* berturut-turut, maka bot akan dianggap mengalami stuck. Setelah itu, bot akan menjalankan strategi pemulihan, seperti membalik arah orbit dan bergerak menuju tengah arena.

4.2.1.2 Struktur Posisi dan Koordinat

Bot Orion menggunakan sistem koordinat arena untuk menentukan posisi dirinya, posisi musuh, serta arah tujuan pergerakan. Dalam Robocode Tank Royale, arena permainan direpresentasikan dalam bidang dua dimensi dengan sumbu X dan sumbu Y. Koordinat ini sangat penting karena hampir seluruh keputusan bot bergantung pada posisi. Bot harus mengetahui di mana dirinya berada, di mana musuh berada, seberapa jauh jaraknya dari musuh, dan seberapa dekat posisinya dengan dinding arena. Koordinat ini digunakan untuk menentukan posisi bot, menentukan

posisi musuh yang terdeteksi, menghitung jarak antara bot dan musuh, memprediksi posisi musuh, menentukan arah menuju tengah arena, dan memeriksa bot apakah berada dekat dinding

Variabel posisi yang digunakan adalah sebagai berikut:

- `currentX` : double
Variabel ini menyimpan posisi bot saat ini pada sumbu X. Nilai ini digunakan untuk menghitung jarak bot terhadap musuh, posisi relatif terhadap dinding kiri dan kanan, serta perubahan posisi dari *turn* sebelumnya.
- `currentY` : double
Variabel ini menyimpan posisi bot saat ini pada sumbu Y. Nilai ini digunakan untuk menghitung posisi bot terhadap dinding atas dan bawah arena, serta mendukung proses deteksi stuck.
- `enemyX` : double
Variabel `enemyX` menyimpan estimasi posisi musuh pada sumbu X berdasarkan hasil *scanning*. Informasi ini digunakan untuk mengarahkan *radar*, *gun*, dan body bot.
- `enemyY` : double
Variabel `enemyY` menyimpan estimasi posisi musuh pada sumbu Y. Bersama dengan `enemyX`, variabel ini digunakan untuk menentukan lokasi musuh secara lengkap pada arena.
- `predictedX` : double
Variabel ini menyimpan hasil prediksi posisi musuh pada sumbu X. Prediksi dilakukan berdasarkan arah gerak musuh, kecepatan musuh, dan waktu prediksi sederhana.
- `predictedY` : double
Variabel ini menyimpan hasil prediksi posisi musuh pada sumbu Y. Nilai ini digunakan agar bot tidak hanya menembak posisi musuh saat ini, tetapi juga mengarahkan tembakan ke posisi yang diperkirakan akan ditempati musuh.
- `centerX` : double
Variabel `centerX` menyimpan koordinat titik tengah arena pada sumbu X. Nilainya dapat diperoleh dari setengah lebar arena.
- `centerY` : double
Variabel `centerY` menyimpan koordinat titik tengah arena pada sumbu Y. Nilainya dapat diperoleh dari setengah tinggi arena.

4.2.1.3 Struktur Data Musuh

Selain menyimpan posisi bot sendiri, Orion juga membutuhkan data mengenai musuh. Data musuh diperoleh melalui *event* `ScannedBotEvent` ketika *radar* berhasil

mendeteksi bot lawan. Informasi musuh digunakan sebagai dasar pengambilan keputusan greedy. Bot tidak menyimpan riwayat panjang semua gerakan musuh, melainkan hanya menggunakan informasi terbaru yang relevan untuk menentukan aksi terbaik pada *turn* tersebut.

Data musuh yang digunakan meliputi:

- enemyDistance : double
Variabel ini menyimpan jarak antara bot Orion dan musuh. Jarak menjadi salah satu faktor terpenting dalam strategi Orion karena menentukan keputusan pergerakan dan kekuatan tembakan. Jika musuh berada jauh, bot akan mendekat. Jika musuh berada pada jarak sedang, bot akan mengorbit. Jika musuh terlalu dekat, bot akan menjauh.
- enemyDirection : double
Variabel ini menyimpan arah relatif menuju musuh. Nilai ini digunakan untuk mengarahkan body bot, menentukan arah orbit, dan mengatur strategi pergerakan.
- enemyEnergy : double
Variabel ini menyimpan energi musuh yang terdeteksi. Informasi energi musuh digunakan untuk menentukan apakah bot perlu bermain agresif atau tetap menjaga jarak. Jika energi musuh rendah dan energi bot masih tinggi, Orion dapat memilih untuk menekan musuh dengan bergerak lebih agresif. Sebaliknya, jika energi bot rendah, Orion akan mengurangi kekuatan tembakan agar tidak cepat kehabisan energi.
- enemySpeed : double
Variabel ini menyimpan kecepatan gerak musuh. Data ini digunakan dalam proses prediksi posisi musuh. Musuh yang bergerak cepat membutuhkan prediksi posisi agar tembakan tidak hanya diarahkan ke posisi saat ini.
- enemyDirectionMove : double
Variabel ini menyimpan arah gerak musuh. Arah gerak tersebut digunakan untuk memperkirakan posisi musuh pada beberapa *turn* berikutnya.

4.2.1.4 Struktur Data Pergerakan

Struktur data pergerakan digunakan untuk menentukan arah dan jarak gerak bot. Bot Orion tidak bergerak secara acak, melainkan memilih gerakan berdasarkan aturan greedy yang telah ditetapkan.

Variabel yang digunakan dalam proses pergerakan:

- moveDirection : double
Variabel ini menyimpan arah tujuan gerak bot. Nilainya dapat berupa arah menuju musuh, arah menjauh dari musuh, arah orbit, atau arah menuju tengah arena.

- moveDistance : double

Variabel ini menyimpan jarak yang akan ditempuh bot pada *turn* tertentu. Jarak gerak disesuaikan dengan kondisi permainan. Misalnya, ketika bot stuck atau dekat dinding, nilai gerak dibuat cukup besar agar bot segera keluar dari posisi berbahaya.

- directionToCenter : double

Variabel ini menyimpan arah dari posisi bot saat ini menuju titik tengah arena. Arah ini digunakan ketika bot perlu keluar dari posisi dinding atau sudut arena.

- isNearWall : boolean

Variabel ini digunakan untuk menandai apakah bot berada dekat dinding arena. Bot dianggap dekat dinding apabila jaraknya terhadap salah satu batas arena lebih kecil dari nilai tertentu. Jika isNearWall bernilai true, maka prioritas gerakan akan berubah. Bot tidak lagi fokus menyerang musuh, tetapi bergerak ke arah tengah arena terlebih dahulu

- isStuck : boolean

Variabel ini digunakan untuk menandai apakah bot sedang mengalami stuck. Jika isStuck bernilai true, maka bot akan menjalankan strategi pemulihan dengan membalik arah orbit dan bergerak ke arah yang lebih aman.

4.2.1.5 Struktur Data Penembakan

Struktur data penembakan digunakan untuk menentukan apakah bot harus menembak, seberapa besar kekuatan peluru yang digunakan, dan ke mana *gun* harus diarahkan. Penembakan pada Orion tidak dilakukan secara sembarangan. Bot hanya menembak jika *gun* sudah cukup mengarah ke target dan kondisi *gun* memungkinkan untuk menembak.

Variabel yang digunakan adalah sebagai berikut:

- firePower : double

Variabel ini menyimpan besar kekuatan peluru yang akan ditembakkan. Nilainya ditentukan berdasarkan jarak musuh, energi bot, dan energi musuh. Jika musuh dekat, firePower dibuat lebih besar karena peluang mengenai target lebih tinggi. Jika musuh jauh, firePower dibuat lebih kecil agar energi tidak terbuang percuma.

- gunTurn : double

Variabel ini menyimpan besar sudut yang dibutuhkan gun untuk mengarah ke target. Nilai ini digunakan untuk menentukan apakah tembakan sudah cukup akurat. Jika nilai gunTurn masih terlalu besar, berarti gun belum mengarah dengan baik, sehingga bot sebaiknya menunda tembakan.

- gunHeat : double

Variabel ini menunjukkan kondisi panas senjata. Dalam Robocode, bot tidak dapat menembak ketika gun masih panas. Oleh karena itu, bot hanya menembak jika gunHeat bernilai 0.

- aimTolerance : double

Variabel ini menyimpan batas toleransi sudut tembakan. Toleransi ini disesuaikan dengan jarak musuh. Jika musuh dekat, toleransi dapat dibuat lebih besar karena peluang tembakan mengenai target masih cukup tinggi. Jika musuh jauh, toleransi harus lebih kecil agar tembakan lebih akurat.

4.2.1.6 Struktur Data Event

Dalam Robocode Tank Royale, bot tidak hanya berjalan berdasarkan loop utama, tetapi juga merespons berbagai event yang terjadi selama permainan. Event tersebut dapat memengaruhi keputusan bot secara langsung.

Beberapa event penting yang ditangani bot:

- onScannedBot

Event ini terjadi ketika radar berhasil mendeteksi bot musuh. Event merupakan inti dari strategi Orion karena sebagian besar keputusan bot diambil setelah musuh terdeteksi. Pada event ini, bot akan menghitung jarak musuh, menentukan arah musuh, mengunci radar ke musuh, mengecek apakah bot mengalami stuck, memilih besar peluru, memprediksi posisi musuh, mengarahkan gun, dan menembak jika kondisi memungkinkan.

- onHitWall

Event ini terjadi ketika bot menabrak dinding arena. Ketika event ini terjadi, bot akan membalik arah orbit dan bergerak menuju tengah arena. Tujuannya adalah agar bot tidak terus berada di dekat dinding, karena posisi tersebut berisiko membuat bot sulit bergerak dan mudah ditembak.

- onHitBot

Event ini terjadi ketika bot bertabrakan dengan bot lain. Pada strategi Orion, tabrakan tidak selalu menjadi tujuan utama. Jika bot bertabrakan dengan musuh, bot perlu mengevaluasi apakah harus menjauh atau tetap menekan musuh. Jika musuh lemah dan energi Orion masih tinggi, bot dapat tetap agresif. Namun, jika kondisi tidak menguntungkan, bot lebih baik menjauh untuk menjaga jarak aman.

- onHitByBullet

Event ini terjadi ketika bot terkena peluru musuh. Ketika terkena peluru, bot perlu segera mengubah pola gerakan agar tidak mudah ditembak berulang kali dari arah yang sama. Respons yang dapat dilakukan adalah membalik arah orbit, bergerak menyamping, atau menjauh dari posisi sebelumnya.

4.2.2 Hubungan Struktur Data dengan Strategi Greedy

Struktur data yang digunakan pada bot Orion memiliki hubungan langsung dengan penerapan algoritma greedy. Setiap atribut dan variabel digunakan untuk mengevaluasi kondisi lokal pada saat itu, kemudian memilih aksi terbaik berdasarkan aturan yang telah dibuat.

Struktur Data	Fungsi dalam Strategi Greedy
enemyDistance	Menentukan apakah bot harus mendekat, mengorbit, menjauh, atau menembak dengan power tertentu
enemyEnergy	Menentukan tingkat agresivitas dan besar tembakan
botEnergy	Menentukan apakah bot dapat menyerang agresif atau harus menghemat energi
orbitDirection	Menentukan arah orbit agar bot tetap bergerak dinamis
isNearWall	Menentukan apakah bot harus memprioritaskan kembali ke tengah arena
isStuck	Menentukan apakah bot perlu menjalankan strategi pemulihan
gunTurn	Menentukan apakah gun sudah cukup akurat untuk menembak
gunHeat	Menentukan apakah bot secara teknis dapat menembak.
firePower	Menentukan besar energi yang

	digunakan untuk menyerang
predictedX & predictedY	Menentukan titik target tembakan berdasarkan prediksi posisi musuh

4.2.3 Efisiensi Penggunaan Struktur Data

Struktur data yang digunakan pada bot Orion tergolong sederhana dan efisien. Bot tidak menggunakan struktur data kompleks seperti tree, graph, priority queue, atau penyimpanan riwayat panjang. Hal ini sesuai dengan kebutuhan permainan real-time, di mana setiap bot harus mengambil keputusan dengan cepat dalam waktu turn yang terbatas.

Namun, struktur data yang sederhana juga memiliki keterbatasan. Karena bot tidak menyimpan riwayat pergerakan musuh secara lengkap, prediksi yang dilakukan masih bersifat sederhana. Bot hanya memperkirakan posisi musuh berdasarkan arah dan kecepatan saat ini. Jika musuh melakukan perubahan arah secara tiba-tiba, prediksi dapat menjadi kurang akurat. Meskipun demikian, untuk strategi greedy rule-based, struktur data ini sudah cukup efektif karena mendukung pengambilan keputusan cepat dan responsif.

4.3 Analisis

4.3.1. Pengujian dan Analisis Performa Strategi Greedy

4.3.1.1 Pengujian Algoritma Rule-Based melawan Algoritma Adaptif

Kondisi: Pertarungan 1 vs 1 antara bot Orion dan bot AllMight

Hasil: Orion menang melawan AllMight tapi dengan skor yang tidak terlalu jauh

Analisis: Dalam Pengujian ini Strategi Algoritma *Rule-Based* berjalan optimal karena mampu melampaui kemampuan adaptif dari lawannya yang dimana memiliki keunggulan dalam hal bertahan hidup, menunjukkan Strategi

Rule-Based pada Orion dapat berjalan sebagai pemburu dan eksekutor secara baik

4.3.1.2 Pengujian menghadapi Strategi Algoritma dengan fokus pada gerakan yang tidak menentu

Kondisi: Pertarungan 1 vs 1 antara bot Orion dengan bot Velocity

Hasil: Orion menang telak terhadap bot dengan fokus pada Strategi pergerakan atau Movement

Analisis: Strategi Greedy berjalan optimal pada kasus ini, karena implementasi dari predictive aiming berjalan dengan baik melawan bot yang berfokus pada gerakan, hal ini ditunjukkan dengan kemenangan telak oleh bot Orion

4.3.1.3 Pengujian menghadapi Strategi Algoritma dengan fokus pada gerakan Spin atau Mengorbit

Kondisi: Pertarungan 1 vs 1 antara bot Orion dengan Spinbot

Hasil: Orion menang telak terhadap bot dengan strategi yang berfokus pada pergerakan Mengorbit atau Spin

Analisis: Strategi Greedy berjalan optimal pada kasus ini, namun kasus ini membuka titik lemah pada prediksi pergerakan musuh pada Orion, dimana masih sering terjadi tidak kena sasaran pada tembakan terhadap musuhnya, yang mengakibatkan beberapa jumlah energi terbuang. Dan kasus ini juga menunjukkan masih ada kekurangan pada adaptasi terhadap kondisi energi bot itu sendiri.

4.3.1.4 Pengujian menghadapi Strategi Algoritma dengan fokus pada Pergerakan Linear Kearah Pojok

Kondisi: Pertarungan 1 vs 1 antara bot Orion dan bot Wall

Hasil: Orion menang telak terhadap bot dengan strategi yang berfokus pada pergerakan linear kearah corner

Analisis: Strategi Greedy berjalan optimal pada kasus ini, perbedaan skor yang besar menunjukkan eksekusi yang baik dari Algoritma Orion, namun pada kasus ini justru memperkuat analisis terkait kekurangan pada kemampuan memprediksi pergerakan musuh, dimana jika musuh bergerak secara linear dan berada pada kecepatan tinggi, prediksi yang ada justru tertinggal jika dibandingkan dengan pergerakan sebenarnya dari bot musuh.

4.3.2 Ringkasan Hasil Pengujian

Kondisi Pengujian	Hasil Strategi Greedy	Optimal
1 vs 1 Melawan Strategi Greedy Adaptif	Orion menang dengan selisih tipis	Ya
1 vs 1 Melawan Strategi Greedy Pergerakan Yang Tidak Menentu	Orion Menang dengan telak	Ya
1 vs 1 Melawan Strategi Greedy Spin atau Mengorbit	Orion menang dengan telak	Ya
1 vs 1 Melawan Strategi Greedy Pergerakan Linear	Orion menang dengan telak	Ya

BAB V

Kesimpulan dan Saran

5.1 Kesimpulan

Berdasarkan hasil perancangan, implementasi, dan pengujian, algoritma greedy dapat digunakan untuk membangun bot Robocode Tank Royale yang mampu mengambil keputusan secara cepat dan responsif. Bot memilih aksi terbaik berdasarkan kondisi saat itu, seperti jarak musuh, energi bot, posisi terhadap dinding, serta kondisi senjata.

Empat bot yang dibuat memiliki strategi greedy yang berbeda. Orion menggunakan strategi rule-based yang seimbang, AllMight menggunakan strategi target locking, Ares menggunakan strategi agresif berbasis ramming, dan Aegis menggunakan strategi defensif untuk bertahan hidup. Perbedaan heuristic ini menunjukkan bahwa strategi greedy dapat diterapkan dengan berbagai pendekatan sesuai tujuan masing-masing bot.

Dari hasil perbandingan, Orion dipilih sebagai bot utama karena memiliki strategi yang paling seimbang antara menyerang, bertahan, menjaga jarak, menghindari dinding, dan mengatur daya tembak. Hasil pengujian juga menunjukkan bahwa Orion mampu menang dalam beberapa skenario pertarungan melawan bot dengan pola permainan yang berbeda.

Walaupun strategi greedy tidak selalu menghasilkan solusi global terbaik, pendekatan ini cukup efektif untuk permainan real-time seperti Robocode Tank Royale. Dengan aturan yang tepat dan akurat, bot dapat mengambil keputusan dengan cepat dan tetap kompetitif dalam pertandingan.

5.2 Saran

Berdasarkan hasil implementasi dan pengujian, bot Orion masih dapat dikembangkan terutama pada bagian prediksi posisi musuh. Prediksi yang digunakan masih sederhana, sehingga tembakan dapat meleset ketika musuh bergerak cepat atau berubah arah secara tiba-tiba.

Selain itu, setiap bot alternatif juga masih dapat ditingkatkan sesuai karakter strateginya. AllMight dapat dibuat lebih efektif pada kondisi 1 vs 1, Ares dapat diberi mekanisme penghindaran dinding yang lebih baik, dan Aegis dapat dibuat lebih aktif menyerang agar tidak terlalu defensif.

LAMPIRAN

Link Repositori: https://github.com/N16Akbar/Tubes_BinTankLima

No.	Poin	Ya	Tidak
1	Bot dapat dijalankan pada Engine yang sudah dimodifikasi asisten.	✓	
2	Membuat 4 solusi greedy dengan heuristic yang berbeda.	✓	
3	Membuat laporan sesuai dengan spesifikasi.	✓	
4	Membuat video bonus dan diunggah pada Youtube.		✓

DAFTAR PUSTAKA

Institut Teknologi Sumatera. (2026). 02b - Algoritma Greedy (Bagian 1).pdf. Pertemuan 2: Algoritma Exhaustive Search dan Algoritma Greedy, IF25-21013 Strategi Algoritma.

<https://kuliah2.itera.ac.id/mod/resource/view.php?id=3470>

Vazirani, V. V., & Williamson, D. P. (2011). *Greedy algorithms and local search*. Dalam *The Design of Approximation Algorithms*. Cambridge University Press.

Hensman, A. (2007). *Evaluation of Robocode as a teaching tool for computer science*. Technological University Dublin.

Robocode Dev Team. Robocode Tank Royale Documentation.

<https://robocode-dev.github.io/tank-royale/>